Towards AI-driven Optimization of Robust Probing Model-compliant Masked Hardware Gadgets Using Evolutionary Algorithms

David S. Koblah[1], Dev M. Mehta[2], Mohammad Hashemi[2], Fatemeh Ganji[2] and Domenic Forte[1]

[1] University of Florida, Gainesville, USA, dkoblah@ufl.edu,dforte@ece.ufl.edu
[2] Worcester Polytechnic Institute, Worcester, USA,
dmmehta2@wpi.edu,mhashemi@wpi.edu,fganji@wpi.edu

Abstract. Side-channel analysis (SCA) is a persistent threat to security-critical systems, enabling attackers to exploit information leakage. To mitigate its harmful impacts, masking serves as a provably secure countermeasure that performs computing on random shares of secret values. As masking complexity, required effort, and cost increase dramatically with design complexity, recent techniques rely on designing and implementing smaller building blocks, so-called "gadgets." Existing work on optimizing gadgets has primarily focused on latency, area, and power as their objectives. To the best of our knowledge, the most up-to-date ASIC-specific masking gadget optimization frameworks require significant manual effort. This paper is inspired by previous work introducing open-source academic tools to leverage aspects of artificial intelligence (AI) in electronic design automation (EDA) to attempt to optimize and enhance existing gadgets and overall designs. We concentrate on evolutionary algorithms (EA), optimization techniques inspired by biological evolution and natural selection, to find optimal or near-optimal solutions. In this regard, our goal is to improve gadgets in terms of power and area metrics. The primary objective is to demonstrate the effectiveness of our methods by integrating compatible gates from a technology library to generate an optimized and functional design without compromising security. Our results show a significant reduction in power consumption and promising area improvements, with values reduced by 15% in some cases, compared to the naïve synthesis of masked designs. We evaluate our results using industry-standard synthesis and pre-silicon side-channel verification tools.

Keywords: Side Channel Analysis · Masking · Evolutionary Algorithms · Gadgets · Artificial Intelligence

1 Introduction

Side-channel vulnerabilities have been studied extensively for over two decades since the seminal work by Kocher et al. introduced their devastating impact on security-critical implementations cf. [KJJ99]. The fundamental premise of side-channel analysis (SCA) is that an adversary can observe and measure physical effects to extract sensitive information during execution, where one of the common types of measurements is power consumption and its close counterpart, electromagnetic (EM) emanation [KJJ99, GMO01]. As new side channels and exploitation methods continue to emerge, countermeasures have been developed, with *masking* being predominantly studied thanks to its formal and sound security foundation [CJRR99]. Masking schemes can be likened to techniques of secret sharing, where secret values are randomly decomposed into *shares* [GSF13]. Operations on these shares are executed to meet a specific security criterion.

Despite ongoing efforts to enhance the efficiency and security of hardware masking schemes, inaccuracies and design flaws have often limited their adoption. In this regard, unintentional physical effects, e.g., glitches, transitions, or coupling, and architectural conditions (parallelism and pipelining, for instance) account for the insecurity of physical instantiation of the masking scheme regardless of their sound theoretical security proofs [KMMS21]. As a response to this, masking of design building blocks, so-called "gadgets", has been introduced to ease the burden of such enormous engineering and error-prone tasks [RBN+15]. One key aspect is the secure composition of gadgets known to be a non-trivial problem relying on security notions and properties; for instance, *probe-isolating non-interference (PINI)* enables efficient and secure compositions for multi-input, multi-output gadgets and trivially secure linear operations, i.e., XORing [CS20a]. To integrate composable PINI gadgets into arbitrary design architectures, an electronic design automation(EDA) tool should be able to handle the task without violating security requirements. While most EDA tools have traditionally focused on optimizing speed, area, and power, they often fail to consider security as a fundamental design constraint as it necessitates a comprehensive, system-wide perspective [FSG23]. Industry-standard tools such as Synopsys Design Compiler [Syn23] are capable of synthesizing designs. However, additional compilation constraints to achieve optimal results may lead to the loss of masking security. Such a trade-off is unsatisfactory, especially when security is paramount in side-channel resilient benchmarks. As a result, add-ons have been bundled into EDA tools to facilitate the implementation of masked circuits.

A prime example of such add-ons is AGEMA [KMMS21], a tool transforming unprotected cryptographic designs into securely masked circuits using different masked gadgets as fundamental building blocks. It explores different processing techniques to achieve this transformation and supports masked gadgets to offer high flexibility concerning the security level, required randomness, latency, and area overhead. Afterward, [MCS22] demonstrated improvements by manual optimization in AGEMA. By doing so, the latency in AES S-box implementations, especially for serial implementations, is reduced by 6×. An added benefit to their method is reduced area consumption due to changes in the number of registers. This line of research has been followed by proposing masked nonlinear components with improved performance [WFP+23] due to latency asymmetry. A very recent tool, AGEMA_FPGA, was introduced as an automated framework for transforming unprotected FPGA netlists into optimized masked designs aligning with the PINI notion [MMG+23]. Compared to AGEMA, primarily dedicated to application- specific integrated circuits (ASICs), it achieves up to 22% fewer registers, up to 64% fewer LUTs, and up to 59% less power consumption in designs while verifying their side channel resilience. Even opportunities for optimization have arisen to improve certain aspects of gadgets without disrupting the secure state. The study [BMN+23] has proposed a security-focused design-space exploration framework that utilizes commercial CAD tools; it analyzes the security-versus-PPA design-space across multiple technology nodes with all their voltage threshold cell options. This provides a better understanding of the relationship between standard cells and static power side-channel vulnerability. Although these tools support (to some extent) the straightforward transformation of unprotected design to a secure one, aspects of the process remain unexplored. First, secure gadgets are designed in a complicated fashion and mostly manually. Second, the optimization techniques available in off-the-shelf tools have been usually avoided to prevent possible leakage. Therefore, the design process produces less-than-ideal gadgets, requiring more resources than necessary.

Contributions. Most prior work has concentrated on optimizing Boolean circuit representations or developing gadgets with reduced randomness and latency [CGLS20, KM22a, SM21, BBP+16]. They, however, often neglect straightforward optimizations within gadgets and their composition that could offer additional efficiency gains. Compared to its most relevant work COMPRESS [CGM+24], we focus on power and area optimization.

More concretely, we aim to optimize the gadgets using commercial and open-source state-of-the-art tools with the integration of AI in the framework. Our study aims to demonstrate the effectiveness of a combination of methods borrowed from electronic design for optimizing masked gadgets, showing significant reductions in power consumption and area. By employing technology mapping, functionally reduced and inverter graphs (FRAIGs), and retiming, our framework achieves as much as 59% reduction in power consumption and a 57% reduction in area compared to the naïve synthesis approach for masked designs. Building on this, we further enhance the optimization of existing gadgets, e.g., ones proposed in AGEMA [KMMS21]. We noted a lack of control in certain optimization scenarios, particularly concerning retiming. The optimization tool, ABC [BM10], functioned somewhat as a "black box", representing a somewhat uninterpretable solution.

In response, we leverage AI-based optimization to provide explainable solutions with precisely defined objectives tailored toward total power and area. The integration of AI-driven techniques, particularly advanced Genetic Algorithms (GAs) [Sam76], in circuit design optimization is becoming increasingly vital in the development of cryptographic hardware [SSM15, BW09]. Nevertheless, their application in the development of masked circuits, especially gadgets, has been overlooked. In fact, AI methodologies could potentially enhance the pre-silicon design phase by optimizing circuit complexity, power, and signal delay, directly influencing the hardware's side-channel resilience. For instance, the adaptive genetic algorithm (AGA) [JG99] focuses on minimizing gate counts, which may reduce potential leakage points that could be exploited in side-channel attacks. Similarly, cell crossover techniques [BW09] provide a more nuanced genetic search process, allowing for precise adjustments to the circuit's layout that preemptively address security vulnerabilities. Applying these AI-driven optimizations, coupled with pre-silicon side-channel resilience techniques, early in the design process ensures that the final designs are not only functionally correct, but also less demanding in terms of post-silicon modifications [BBYS22, KS22]. To ensure the security of the output from the tool, we utilize the VERICA [RBFSG22], which is tailored for pre-silicon security analysis.

The AI-driven technique offers a comprehensive exploration of numerous potential design solutions compared to the FRAIG-based method. While the GA is unsupervised, its strength lies in exploring various evolutionary combinations to identify suitable solutions. By controlling various hyperparameters, we can prioritize weighted objectives and guide evolution. Our AI model consistently maintains PINI security and, in some instances, maximizes optimization to 68.1% for power efficiency and 68.7% for area reduction. Using the same model, we apply this to selected COMPRESS [COM] artifacts to ensure that the benefits of trivial composability in the PINI framework translate effectively to selected AES and Skinny cryptographic designs.

2 Background

2.1 Masking and Gadgets

Boolean masking. In essence, the goal of a masking scheme is to ensure security at an established order d, under assumptions concerning the leakage behavior of the targeted device. A prevalent approach to masking is Boolean secret sharing, which uses binary addition to distribute sensitive data. In this context, a secret variable $x \in GF(2^m)$ is split into $d+1$ shares $(x_1, \ldots, x_{d+1})$ with the property $x = \bigoplus_{i=1}^{d+1} x_i$. Uniformity is assured by selecting shares $x_1, \ldots, x_d$ from a uniform distribution and determining $x_{d+1} = x \oplus \bigoplus_{i=1}^{d} x_i$ (known as the *correctness property*).

Probing security. To abstract and formalize the behavior of masked circuits and model an adversary's ability to extract information, various models have been proposed over time, each offering different trade-offs between simplicity and accuracy. One of the foundational

models is the d-probing model, introduced by Ishai et al. [ISW03]. According to this model, an adversary is allowed to observe the distribution of up to d internal wires within a circuit. A masked circuit is said to achieve d-probing security if an adversary cannot gain any information about the sensitive value x being processed.

In the paper [GF21], gadgets are central to any masking scheme, acting as the crucial elements that transform each gate in the original circuit into non-leaky clusters while preserving the circuit's intended functionality. This model forms the baseline for probing the security of masked circuits and is known as the *d-probing model*. A gadget G is d-probing secure if the adversary's output is independent of sensitive variables under fresh input sharings. Thus, the security order d represents the number of probes [CS20a]. Any implementation with $d+1$ shares is d-probing secure for linear functions [CS20a]. Physical implementations of masked circuits can suffer from effects such as glitches and coupling, which indirectly lead to sensitive shares being leaked during SCA [CS20b, FGDP+18]. These effects can be modeled using the glitch-robust probing model, which is an extension to the d-probing model [FGDP+18]. For this model, glitches are modeled using extended probes where any probe on the output variable extends to its corresponding inputs. Similarly, transition and coupling effects are taken into account by extended probes such that probes allow access to memory recombination and adjacent wires, respectively.

Robust probing model and glitch-extended probes. The traditional probing model, primarily suited for software implementations, fails to capture key physical effects in hardware, e.g., transitions (memory recombinations), glitches (combinational recombinations), and wire coupling (routing recombinations) cf. [KMMS21]. To address this limitation, the robust probing model was introduced in [FGDP+18], offering a more accurate yet efficient way to verify masked hardware designs. Unlike the traditional model, which assumes wire values remain stable during evaluation, the robust model incorporates registers and extended probes to account for leakage caused by hardware-specific behaviors. Notably, glitch-extended probes can observe recombined signals caused by differences in gate delays, allowing a single probe to capture the joint leakage of all contributing stable signals. This model replaces a probe on a wire with a set of probes at its contributing register outputs and primary inputs, effectively tracing the path from stable sources to the observed point.

Gadgets and composability. Designing masked circuits is a tedious task with significant engineering complexity and a high risk of errors. To tackle that, masking circuit components, commonly referred to as gadgets, have been considered in the literature [RBN+15, BBD+16, KSM22, KMMS21]. Creating secure gadgets for fundamental non-linear operations, such as AND, enables efficient masking of any digital logic circuit represented using AND-XOR structures. For instance, the Trichina AND [Tri03] ensures security at the gate level for $\lambda = 1$, indicating security against a single probe. It is specifically designed to manage two 2-share inputs, which is critical for its security assurances. The ISW AND [ISW03] lays the foundation for threshold implementations, providing a robust basis for crafting higher-order protection schemes. Nevertheless, the composition of individually secure gadgets does not automatically result in a secure overall circuit. Therefore, advanced security models and properties are crucial for analyzing and ensuring the composability of masked gadgets.

Strong non-interference (SNI) and non-interference (NI) models address the secure composition of masked circuits, particularly when building systems from gadgets [KSM22, BBD+16, BBD+15]. NI ensures that security is maintained across the entire circuit. However, partial information can still leak due to probe propagation, where probing downstream gadgets can expose information from upstream ones. SNI strengthens this by preventing such propagation at gadget outputs, requiring these outputs to be perfectly simulatable without relying on any partial information. Yet, SNI is limited to single-output gadgets and does not scale well to multi-output ones. To overcome this, PINI has been introduced [CS20a], which confines probes to specific share domains and prevents inter-

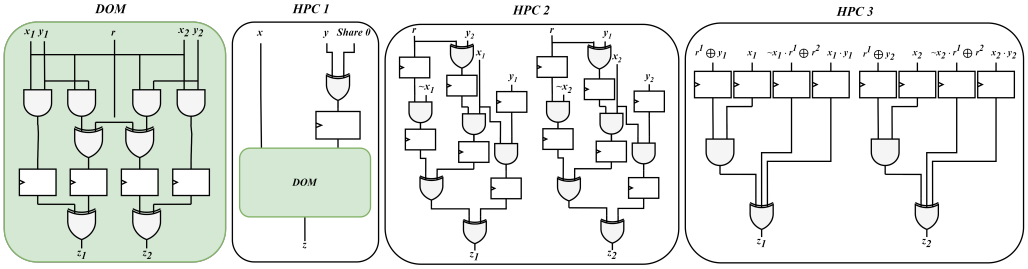

Figure 1: Diagram of the DOM implementation and its use in the HPC 1 gadget, whereas HPC 2 and HPC 3 do not utilize the DOM gadget. Here, x, y, and z represent input and output while r and *share 0* represent the refresh bits. The subscripts represent the shares, and the superscripts represent different refresh bits (see Sections 2.1 and 4).

share information flow. Unlike SNI, PINI supports multi-output gadgets and allows trivial composition of linear functions, making it more suitable for modern, glitch-robust probing model-compliant design.

An example of gadgets not complying with the glitch-robust probing model. DOM multiplication, shown in Figure 1, is an example of a widely used gadget. Initially, cross-product calculations are performed, with randomness introduced to specific products. Precisely, for a *first-order* security level, the following is derived [DCEM18]:

$$p_1 = x_1 y_1,\ p_2 = x_1 y_2 \oplus r_1,\ p_3 = x_2 y_1 \oplus r_1,\ p_4 = x_2 y_2. \tag{1}$$

Subsequently, the terms p_i from Eqn (1) are stored in a register and then proceed to a compression phase. Here, the $(d+1)^2$ shares are transformed to $(d+1)$ shares for the output z_i. For example, $z_1 = p_1 \oplus p_2$ and $z_2 = p_3 \oplus p_4$. DOM multiplication requires independent input shares and necessitates an additional clock cycle in the compression layer [GMK16, GMK17, GM17]. Nonetheless, the DOM multiplier remains a frequently adopted masking scheme, with its security rooted in the independent power usage of its constituent functions.

The security of DOM does not hold up well under the glitch-robust probing model. The security of DOM is reduced to NI, which does not allow for the secure composition of gadgets under probe propagation scenarios. Therefore, Faust et al. [FGDP+18] proposed a multiplication gadget with 2 DOMs with 2 registers to make it glitch-robust SNI. This implementation takes 4 cycles with a higher randomness cost. This scheme was improved upon by [CGLS20] with the introduction of hardware private circuits (HPC). They proposed 2 versions of HPC, HPC1 and HPC2, as shown in Figure 1. HPC1 builds upon the DOM approach as in the Faust implementation, but enhances it by reducing both the number of cycles to two and the amount of randomness required. Additionally, it ensures glitch-robust PINI security, providing strong guarantees for composability. HPC2 has the same security with a randomness requirement equal to DOM due to a ground-up design. Lastly, HPC3 improves the latency of the HPC2 while doubling the randomness as a trade-off, as shown in Figure 1.

2.2 VERICA

Introduced in [RBFSG22], VERICA stands out as a tool for verifying masked circuits in the pre-silicon phase. It employs the probing model [ISW03] as one of its primary methodologies, making it especially suited for verifying masked designs. Moreover, VERICA integrates a plethora of other models, such as NI [BBD+15], SNI [BBD+16], PINI [CS20a], and active security principles [DN20]- to name a few.

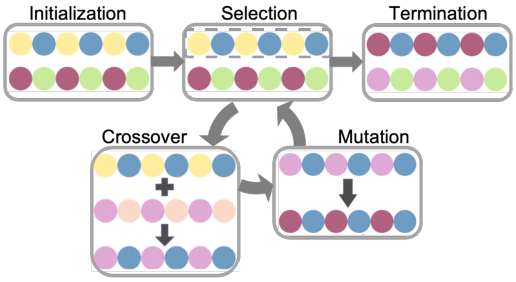

Figure 2: Visualization of genetic algorithm process: initialization of population with individuals represented by colored circles (top-left), selection of parents (top-center), crossover to exchange material for offspring production (bottom-left), mutation or alteration for diversity (bottom-right), and iteration to form a new population until termination criteria are met (top-right)

Another significant advancement lies in VERICA's binary decision diagram (BDD) engine, which is considerably enhanced. Coupled with its modular interface, this tool allows for seamless assessment of varied module versions. This modularity is realized by incorporating a JSON file to annotate share details for the netlist, negating the need to modify the netlist directly. Consequently, consistent ports across diverse implementations exempt users from repeatedly creating new annotations, paving the way for a swift and streamlined experimentation process.

2.3 Evolutionary Algorithms

Genetic algorithms, belonging to the larger class of evolutionary algorithms, are inspired by the process of Darwinian-based natural selection. They operate by evolving a population of candidate solutions over successive generations to find optimal or Pareto solutions to a given problem [BS93]. The key components, including the ones used in this paper, are:
1. **Initialization**: A population of potential solutions is created, often randomly or using specific heuristics.

2. **Selection**: Solutions are evaluated based on a fitness function, and those with higher fitness are more likely to be selected for reproduction.

3. **Crossover**: Pairs of selected individuals exchange genetic information to create new solutions, mimicking the crossover of genetic material in biological reproduction.

4. **Mutation**: Random changes are introduced to some solutions, adding diversity to the population and expanding the solution search space.

5. **Termination**: The algorithm stops when a termination criterion is met, such as reaching a maximum number of generations or achieving a satisfactory solution.

Examples of evolutionary algorithms include *genetic algorithms (GAs)* used in various optimization problems like job scheduling and financial modeling, and *differential evolution* widely used for numerical optimization problems in engineering design, parameter tuning, and robotics. These algorithms are especially suitable for complex, high-dimensional problems where traditional optimization approaches may struggle [DM13].

3 Methodology

The main objective of the first step in our framework is to enhance given masked gadgets through GA with regard to common performance metrics, such as latency, area, and total power. Afterward, we also examine how much improvement can be achieved for

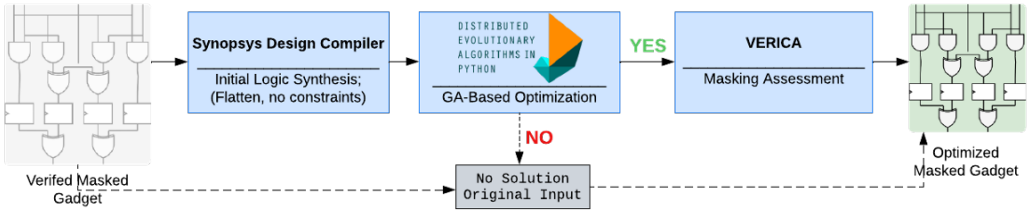

Figure 3: Our framework with its corresponding tools.

the entire design. Our framework is designed to be seamlessly integrated into the ASIC-focused EDA framework, specifically for masked designs. The input benchmark is a register transfer level (RTL) design that has to be verified for side-channel resilience using VERICA [RBFSG22] (see Figure 3). To overcome potential concerns with node repetition and limited search space diversity, we employ the digraph implementation within our DEAP-based pipeline for evolutionary processes. In this representation, the nodes (or so-called vertices) are the logic gates, whereas the connections between gates (fan-in/fan-out) are the edges. This approach allowed crossover and mutation operations to occur between branches and leaves (or nodes) of the original circuit, thereby substantially expanding the search space (see Section 5 for the discussion on string-based graph implementation).

3.1 Genetic Algorithm and its Environment

Non-dominated sorting genetic algorithm (NSGA-II) is the selected GA implementation we use; see Figure 4. It is an efficient multi-objective evolutionary algorithm (MOEA) designed for solving problems with multiple conflicting objectives [DPAM02, DAPM00]. It builds on the original NSGA by enhancing computational efficiency, eliminating parameter tuning, and maintaining population diversity through crowding distance [VPS21]. Elitism ensures that the best solutions are retained, leading to faster convergence and robust exploration of the solution space. NSGA-II ranks solutions based on non-domination [Ses06]. A solution A dominates solution B if A is no worse than B in all objectives and strictly better in at least one. Non-dominated solutions form the Pareto fronts sequentially. Crowding distance preserves diversity along the Pareto front. The crowding distance d_i for solution i is calculated as:

$$d_i = \sum_{k=1}^{M} \frac{f_k^{i+1} - f_k^{i-1}}{f_k^{\max} - f_k^{\min}}, \tag{2}$$

where f_k^{i+1} and f_k^{i-1} are the k-th objective values for the nearest neighbors of solution i, and $f_k^{\max}$ and $f_k^{\min}$ are the maximum and minimum values of the k-th objective in the population [MZS+23, RNJ05].

NSGA-II employs an elitist strategy to ensure that the best solutions are carried over to the next generation, improving or maintaining the Pareto front's quality over time [GS20]. Distributed evolutionary algorithms for Python (DEAP) is the foundation of our optimization framework that integrates the capabilities of Python with evolutionary computation components [FDRG+12]. It supports the application of existing and novel ideas in evolutionary algorithms, offering simplicity and explicitness through its streamlined and transparent algorithms.

3.1.1 Initialization Phase

The optimization process begins by setting up the initial environment for the GA. The function `main_func(path, pop_size)` takes two key inputs: `path`, which specifies the

Table 1: Evaluating Complexity Value ecv_i, Evaluating Power Value (epv_i), and Evaluating Signal Delay Value (edv_{jk}) for the NanGate 45nm technology, according to [BW09].

Gate Type	ecv_i	ecv_i	edv_j	edv_k
NAND	14	17	13	16
NOR	14	17	13	16
XNOR	18	16	14	14
NOT2	12	18	12	17
WIRE	10	14	16	12
AND	16	15	15	13
OR	16	15	15	13
XOR	18	16	14	14

file path to the input circuit design, and `pop_size`, which defines the size of the population to be evolved. This population size determines how many candidate solutions will evolve throughout the optimization process. The input benchmark is instantiated by reading the design via the `cg.from_file` function. We initialize the `BlackBox` model representing a flip flop (FF) with inputs `CK` (clock) and `D` and produce an output `Q`. Once instantiated, the benchmark is converted into a gate-level Verilog netlist using the `circuit_to_verilog_modified` function. This representation allows for the evaluation of the input benchmark's fitness.

The function `f2_with_tech(netlist)` calculates the initial fitness values (power, delay, area). This function computes baseline values stored in a threshold variable. It is derived from the PPA fitness function described below. An auxiliary function `f2_with_power_delay(netlist)` only evaluates power and delay, although these results are used to create an additional set of elite individuals during the evolutionary process.

Multi-objective Fitness Function To evaluate the secure design, we have implemented the fitness function calculation according to Equation 3, which is inspired by [BW09].

$$F = \left(\sum_{i \in N} ecv_i\right) \alpha_c + \left(\sum_{i \in N} epv_i\right) \alpha_p + \left(\sum_{j \in \text{Cols}} \left(\min_{k \in \text{Rows}} edv_{jk}\right)\right) \alpha_d \tag{3}$$

In Equation 3, the evaluating complexity value ecv_i represents the complexity of each cell in the circuit. Higher values indicate more complex cells. The evaluating power value (epv_i) measures the power usage of each cell, which is crucial for energy efficiency. The evaluating signal delay Value (edv_{jk}) indicates the signal delay for the cell located at column j and row k. Lower values are preferred for faster circuit operation. For NanGate 45nm technology, the information related to ecv_i, epv_i, and edv_{jk} is provided in Table 1. Moreover, weighting coefficients ($\alpha_c, \alpha_p, \alpha_d$) are assigned to the complexity, power, and delay values in the fitness function. These adjust the impact of each component on the overall fitness score.

Algorithm 1 shows a high-level description of our fitness function calculator. In the first step, the fitness function calculator (FFC) parses the netlist N. It also finds the design input I, all FF inputs IFF, the design output O, and all the FF outputs OFF. For each member, $IN \in \{I, OFF\}$, the second step will be repeated until every member of the FF outputs or design input is checked. The second phase procedure depends on the connected gates to IN. If only one gate is connected to the IN, the connected gate is appended to the path of the input, $Path[IN]$, and the output of the gate is appended to the array of the concatenated FF outputs and design input $\{I, OFF\}$. If more than one gate is connected to IN, for each connected gate, a new path with a new index, $Path[IN\_i]$, is generated, and the connected gate is appended to the new indicated path. The rest of this phase is similar to the case when only one gate is connected to the IN. In the third and final step, the fitness function is calculated based on the paths for each input similar to [BW09].

Algorithm 1 A high-level description of our fitness function calculator

```
N ← Netlist.v; I ← Inputs(N); IFF ← InputsFF(N);
O ← Outputs(N); OFF ← OutputsFF(N); Path ← {};
for ∀IN ∈ {I, OFF} : do
    while IN ≠ ∀X ∈ {O, IFF} do
        if IN is connected to one gate then
            Append(Path[IN], gate)
            Append({I, OFF}, Outputs(gate))
        else
            for ∀gate_i ∈ connected_gates : do
                Append(Path[IN_i], gate_i)
                Append({I, OFF}, Outputs(gate_i))
            end for
        end if
    end while
end for
```

3.1.2 Sensitivity Analysis

After the setup phase, the sensitivity of the design is analyzed. The circuit is sequentially unrolled using the `cg.tx.sequential_unroll` function. We perform unrolling to eliminate timing elements, transforming the sequential circuit into a combinational one for analysis [MZM08]. Unrolling is typically used to analyze the temporal behavior of sequential circuits, allowing us to observe how nodes in the circuit behave across multiple clock cycles. The number of unrolling steps depends on the number of timing elements and clock cycles. However, the number of unrolling steps for a design with flip flops increases exponentially to $2^{n_{FF}}$ where n_{FF} is the number of flip flops. Results by Miskov-Zivanov et al. have suggested that unrolling an average of 10 stages per benchmark yields an insignificant error value [MZM08, MZM07]. This reduces computational time and eliminates potential bottlenecks. Therefore, setting the unrolling value appropriately ensures reasonable computational complexity.

The sensitivity is calculated using `cg.props.sensitivity` for each node in the unrolled circuit. Sensitivity measures the switching activity of nodes in the overall circuit. The sensitivity function identifies the nodes with the highest sensitivities, corresponding to their switching activity, and stores them in the list named `assigned_nodes`. High switching activity in a circuit corresponds to frequent changes in the signal states and affects power consumption and timing [WCC09]. For equivalence checking, we utilize the SAT [MS08] and Miter [ASF08] methods.

3.1.3 Crossover and Mutation

We first initialize a custom fitness class, `FitnessMulti`, to minimize PPA using specified weights. The `Individual` class represents a candidate design, and individuals are evolved through selection, mutation, and crossover operations. We perform single-point crossover between two individuals using the `sensitivity_crossover` function, randomly choosing a sensitive node elite set from `assigned_nodes`. The mutation step is handled by the `mutate_individual` function, which randomly alters parts of the individual design with a mutation probability value. The mutation function supports four types of mutations: changing the gate type of a connection, altering the source and target nodes of a connection, adding a new connection, and deleting an existing connection. The possible nodes and gate types are dynamically extracted from the current individual, ensuring that mutations remain within the valid search space. The function works by first selecting a mutation type at random and then applying the corresponding mutation to the individual, either modifying existing connections or introducing new ones.

The core of the GA is an iterative process that evolves gadgets over multiple generations, as seen in Figure 4. First, offspring are selected using the `tools.selRandom` function, selecting the top fixed percentage of the population based on their PPA fitness scores.

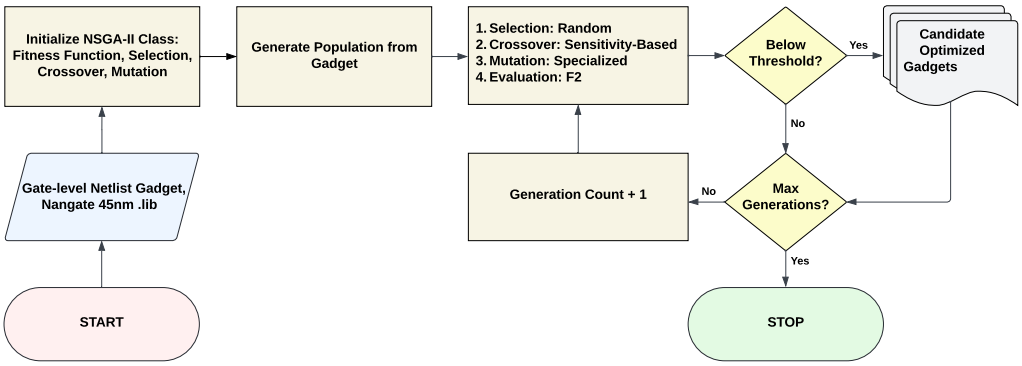

Figure 4: Overview of GA Operations used in the NSGA-II Algorithm

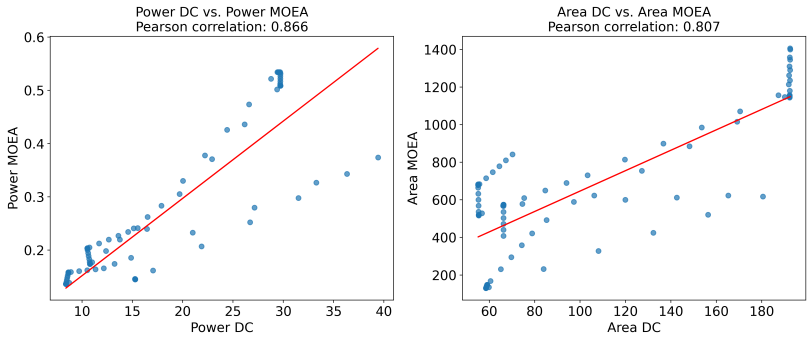

Figure 5: Visualization of Pearson's correlation between Synopsys Design Compiler (DC) and our fitness function (MOEA)

3.1.4 Termination Phase

Fitness evaluation is performed by the `f2_calculate_fitness` function, which re-evaluates individuals whose fitness becomes invalid due to mutation or crossover. The algorithm continues to evolve the population until one of two conditions is met: either a solution that is below the predefined PPA threshold is found, or the maximum number of generations is reached. If the algorithm runs for a predefined number of generations without finding a satisfactory solution, the evolutionary process is restarted. The population may also be regenerated with fresh individuals, and the evolution is repeated. Finally, a gate-level circuit is generated from this optimized design, providing a reference for evaluating and comparing the algorithm's performance. The algorithm using DEAP is detailed in Algorithm 2.

Best fitness vs. tournament-based selection. In our optimization-based approach using GAs, the choice of selection mechanism is critical to balancing convergence speed and solution diversity. We evaluate two prominent selection strategies: *best fitness selection* and *tournament-based selection* [BT96, ZK00, DWZG18]. The best fitness selection involves selecting individuals based solely on their fitness scores to be involved in the next generation. This elitist approach ensures that the highest-performing individuals are consistently propagated to subsequent generations, which can accelerate convergence toward optimal solutions. However, this method carries the risk of premature convergence, as the consistent selection of top performers may reduce genetic diversity and cause the algorithm to become trapped in local optima. Consequently, while the best fitness selection promotes rapid improvement, it may limit the exploration of the broader search space necessary for

Algorithm 2 DEAP-Based Genetic Operations

```
pop ← toolbox.population(n=population size)
CXPB ← Predefined probability that two individuals are crossed.
MUTPB ← Predefined probability of mutating an individual.
offspring ← toolbox.select(pop)
for child1, child2 in zip(offspring[:: 2], offspring[1 :: 2]) do
    if random.random() < CXPB then
        toolbox.mate(child1, child2)
        del child1.fitness.values; del child2.fitness.values
    end if
end for
for mutant in offspring do
    if random.random() < MUTPB then
        toolbox.mutate(mutant)
        del mutant.fitness.values
    end if
end for
invalid_ind ← [ind | ind in offspring | ¬ind.fitness.valid]
fitnesses ← map(toolbox.evaluate, invalid_ind)
for ind, fit in zip(invalid_ind, fitnesses) do
    ind.fitness.values ← fit
end for
pop[:] ← offspring
fits ← [ind.fitness.values[0] | ind in pop]
min_fitness ← min(fits)
```

discovering more diverse and potentially superior solutions.

In contrast, tournament-based selection introduces a competitive element by randomly selecting a subset of individuals (the tournament) and choosing the best performer within this group for reproduction [BT96]. This method maintains higher genetic diversity by giving less fit individuals a chance to contribute to the pool, thereby enhancing the algorithm's ability to explore a broader range of solutions. Additionally, the selection pressure in tournament-based selection can be finely tuned by adjusting the tournament size, offering a balance between exploration and exploitation. However, this flexibility introduces parameter sensitivity, as inappropriate tournament sizes can weaken the selection pressure.

3.2 Synopsys Design Compiler vs. Multi-objective Fitness Function

Applying the fitness function provided in Section 3.1.1, we can assess the optimization of a given design. By PPA fitness, we can evaluate the optimization of a given design independently. This approach is crucial as it enables us to eliminate reliance on external tools during the evolution process. As outlined earlier, we extract power and area values for each cell from a library file, enabling reasonable measurement. The non-linear delay model (NLDM) is a table-based timing model used in very large-scale integration (VLSI) to estimate cell delay and output transition times. It uses lookup tables that map these values as functions of input transition time and output load, allowing accurate and efficient timing analysis. Widely used in static timing analysis (STA) for CMOS circuits, NLDM provides a realistic delay estimate critical to performance optimization [EM11].

Pearson's correlation coefficients for power and area between our multi-objective fitness function and the Synopsys Design Compiler are 0.866 and 0.807, respectively (see Figure5). Pearson's coefficient measures the strength and direction of the linear relationship between two variables, with values closer to 1 indicating a stronger positive correlation [Sed12]. Although Synopsys Design Compiler, which applies NLDM to the NANGATE 45nm library, provides more precise results, our fitness function demonstrates substantial alignment with it. This suggests that our fitness function reliably approximates power, timing, and area metrics despite its relative simplicity.

Table 2: Setting of the genetic operations.

Genetic Operation	Type	Value
Population	Direct Acyclic Graph	100
Selection	Tournament	10 indivs [40×]
	Best	1 indiv [10×]
Crossover	Single Point	0.2, 0.4, 0.8
Mutation	Random, Gaussian	0.2, 0.4, 0.8

3.3 Security Verification using VERICA

While the design compiler generates vital reports, it also "translates" designs obtained in Verilog from the GA process into the technology library-based format required by VERICA. The final benchmark is assessed using VERICA [RBFSG22] again; an ideal result satisfies optimization while retaining its original security characteristics, whereas an unsuccessful design flags the toolchain to perform the evolutionary process again.

4 Experiments & Results

The benchmarks we use in our experiments are obtained from the open-source AGEMA case studies [KMMS21]. To assess the side channel resilience after optimization, we employ VERICA [RBFSG22] to check for adherence to the PINI probing model. To streamline the experimental process, we utilize a shell script to run each tool when required automatically. Additionally, we use Python scripts to generate annotation and configuration files required to designate masking shares for VERICA. For translation/synthesis with the design compiler, we rely on the open-source NanGate 45nm Design Library [Nan]. All experiments are executed on a high-performance computing infrastructure; we access 4 CPU cores with 80 GB of memory. This is supplemented by a single GeForce 2080Ti GPU unit. The parameters used for the evolutionary stages can be seen in Table 2. We also point out that the metrics under consideration are total power and area, whereas optimizing latency and/or randomness is beyond the scope of this work.

Gadget implementations under consideration. We consider COMAR [KM22a] and HPC gadgets, particularly HPC1 and HPC2 [CGLS20], as well as HPC3 [KM22b] for our experiments (see Figure 1). HPC is a specialized type of private circuit designed for the hardware, which is glitch-resistant and enables trivial composition at arbitrary orders [CGLS20]. For example, the HPC1 gadget is based on the refresh-then-multiply technique, where a glitch-robust refresh is added to the DOM to make it a glitch-robust PINI gadget. Composable Gadgets with reused fresh masks (COMAR) enable first-order-secure hardware implementations in the d-probing model under glitches, using only 6 fresh random bits [KM22a]. They ensure secure composability for $d = 1$, minimizing randomness and design errors by constructing 2-input XOR and AND gates, extendable to multiple inputs. COMAR gadgets significantly reduce randomness overhead while maintaining composability and security compared to existing schemes.

4.1 Optimization Using Open-Source Tool: ABC

We start by answering the question of whether optimization options coming with an off-the-shelf design compiler could be used effectively to optimize and improve existing gadgets and overall designs. For this, we focus on technology mapping, retiming, and functionally reduced inverter graphs (FRAIGs). The technology mapping process aims to find the most suitable gate combinations to maintain functionality while optimizing the design. We compare the ABC tool and the Synopsys Design Compiler for this purpose. ABC is our choice because FRAIG allows for the functional representation of networks via improved technology mapping using multiple structural choices [MCJB05]. This allows multiple implementations of suitable gate combinations throughout the design, enabling

the exploration of various options to find the overall best solution. In contrast, Synopsys Design Compiler provides the best possible solutions within specified constraints.

As in the methodology explained in Section 3, the input benchmark is the result of unconstrained synthesis, with the design also verified for side-channel resilience using VERICA [RBFSG22]. The verified design initializes the framework with logic synthesis via Yosys [WGK13], which converts Verilog to Berkeley Logic Interchange Format (BLIF), meeting the input requirements for ABC [BM10]. This format is invaluable because it retains ports, input shares, and output shares of the masked gadgets.

ABC for Optimization ABC provides logic optimization and technology mapping [BM10]. The mapping process identifies the best-fit standard cells [TAM93], with a modified NanGate 45nm library for compatibility with VERICA.

FRAIGs and Retiming FRAIGs improve technology mapping using multiple structural choices [MCJB05]. We employ the FRAIG manager to evaluate multiple design variants using commands: `fraig`, `map`, `fraig_sweep`. Additionally, retiming rearranges registers to enhance timing without altering functionality [MCBP06], adding the textttretime command to our ABC set. The FRAIG implementation also enables functional verification between the input design and its optimized variant.

The post-ABC design is formatted with Synopsys Design Compiler [Syn23] to ensure compatibility with VERICA and generate compilation reports for validation. The final benchmark is reassessed using VERICA [RBFSG22] to confirm that optimization retains security characteristics. If optimization fails, the design reverts to another variant retained by ABC's FRAIG manager, ensuring the synthesis produces an optimal result. Table 3 lists the power and area consumptions of gadgets considered in this study.

According to our observations, retiming shows varied optimization results across different use cases. This is because it can disrupt rules in a sequential design, as retiming changes the placement of registers and flip-flops, which can potentially alter signal timing and affect the design's latency. For HPC1, the optimization difference is minimal, while HPC3 and COMAR gadgets typically have higher overhead with retiming. In contrast, HPC2 gadgets exhibit significant improvements. Focusing on the total power consumption reveals that HPC2 and HPC3 gadgets consistently demonstrate improvements in power consumption, with HPC3 achieving up to a 59% reduction. Area optimization follows similar positive trends, mitigating the overhead from masking schemes while ensuring side-channel resilience.

Timing aims to reduce latency, demonstrated by marginal delay changes in Synopsys Design Compiler reports, while maintaining security. PINI security is mostly consistent, although there are instances where the Design Compiler cannot translate some designs due to corrupted ABC output, particularly for retimed MUX COMAR gadgets. Additionally, the naïve approach shows a loss of PINI security in AND2 and MUX HPC1 gadgets.

Our main conclusion is that ABC achieves significant optimization due to its specialized functionality. However, the limited control over parameters and commands, as well as intermittent PINI security loss, makes ABC an unreliable method for masked gadget optimization (see more results outlined in Appendix A).

4.2 Optimization Using GA

Since off-the-shelf optimization techniques might not enhance the design of gadgets with respect to our metrics without security trade-offs, we shift our focus to GAs. A GA can circumvent the lack of control beyond the command-line functionality provided by open-source tools such as ABC [BM10], preventing improving gadgets to their full extent. Here, we experimentally verify to what extent integrating GA in the masked gadget design flow can boost the optimization process, allow for more custom genetic operations, and better manage optimization parameters. This section reviews characteristics and multiple

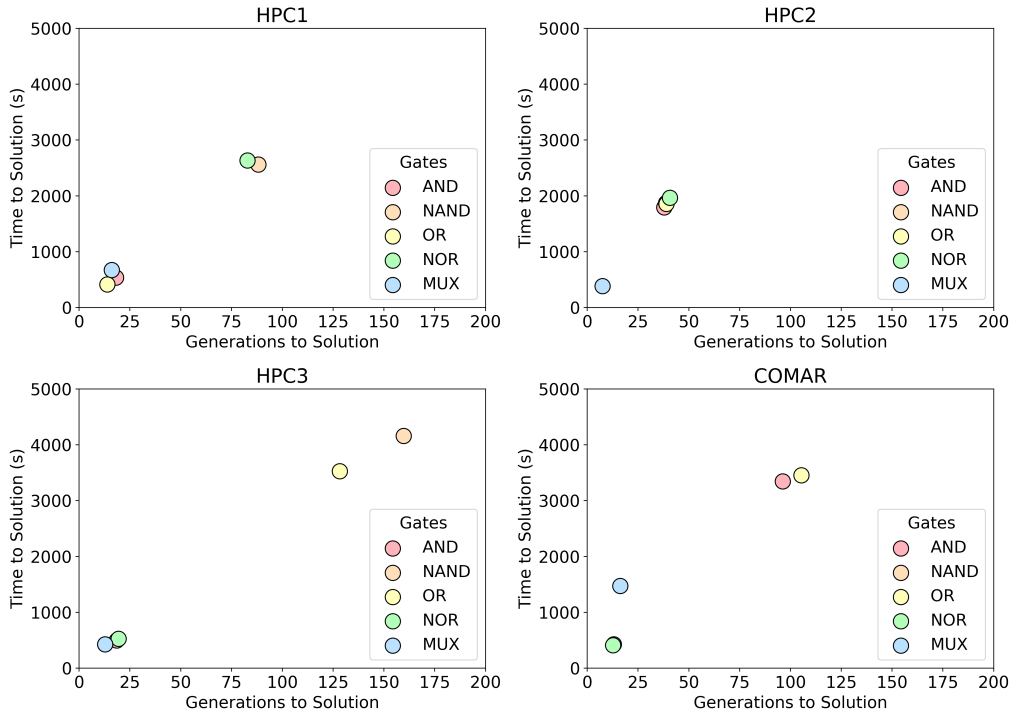

Figure 6: Average number of generations to 1st solution against average time to 1st solution

optimization methods within GA to discuss the most effective one in the context of gadget design.

Time vs. Complexity. The average time to solution is directly influenced by the structural metrics of a given gadget. As shown in Table 3, HPC2 gadgets are the largest, while HPC1 gadgets are the smallest. However, HPC1 and HPC3 gadgets have higher randomness requirements compared to HPC2 gadgets. Our analysis shows that finding solutions for HPC2 gadgets is typically easier and more consistent in terms of time to solution. However, HPC3 and COMAR gadgets show a larger variance for individual gadgets. The OR HPC3 and OR COMAR gadgets exhibit reasonable difficulty in finding solutions, while NAND HPC3 and AND COMAR gadgets are particularly difficult to solve. This variance for HPC1, HPC3, and COMAR gadgets can be attributed to their higher randomness requirements. Although COMAR uses a constant randomness value, it still faces challenges in reaching a reasonable solution, indicating our tool's susceptibility to higher randomness values. Despite HPC3's latency-randomness trade-off compared to HPC2, randomness remains a significant factor in finding solutions.

Generation Count. Another notable observation during the experimentation process is the relationship between generation counts and gadget size. The number of generations needed for a single design depends on the size and complexity of the gadgets. After experimentation, we determined, as observed in Figure 6, that a maximum generation count of 200 was optimal for timely, reachable optimization. In cases where the process failed to yield early viable solutions, we iterated until satisfactory results were achieved. Overall, we conducted 10 iterations, each consisting of 200 generations per gadget.

Table 3: Original gadgets used for experimentation with corresponding power and area obtained using Synopsys Design Compiler

Gadget	Logic Gate	d	Latency (ina, inb)	Random Bits	Power w/o PRNG (μW)	Area w/o PRNG (μm^2)
COMAR	AND	2	1,2	6	15.265	66.234
	NAND	2	1,2	6	15.266	66.234
	OR	2	1,2	6	15.285	66.234
	NOR	2	1,2	6	15.282	66.234
	MUX	2	1,2	6	40.385	184.338
HPC1	AND	2	1,2	2	8.394	58.52
	NAND	2	1,2	2	8.404	58.52
	OR	2	1,2	2	8.643	59.052
	NOR	2	1,2	2	8.633	59.052
	MUX	2	1,2	2	12.887	82.992
HPC2	AND	2	1,2	1	29.690	192.318
	NAND	2	1,2	1	29.726	192.318
	OR	2	1,2	1	29.717	192.584
	NOR	2	1,2	1	29.354	191.786
	MUX	2	1,2	1	17.962	103.740
HPC3	AND	2	1,1	2	10.790	55.328
	NAND	2	1,1	2	10.786	55.328
	OR	2	1,1	2	10.521	55.062
	NOR	2	1,1	2	10.531	55.062
	MUX	2	1,1	2	15.529	70.224

Table 4: Improvements for best fitness selection method. A negative value indicates reduced overhead.

Gadget Type	Logic Gate	Power w/o PRNG (μW)	Area w/o PRNG (μm^2)	Power Reduction [%]	Area Reduction [%]
COMAR	AND	14.844	66.500	-2.760	0.402
	NAND	15.267	66.234	0.003	0.000
	OR	15.208	66.234	-0.503	0.000
	NOR	15.163	66.234	-0.778	0.000
	MUX	40.061	183.806	-0.803	-0.289
HPC1	AND	7.977	58.520	-4.962	0.000
	NAND	8.281	58.520	-1.460	0.000
	OR	8.749	59.584	1.231	0.901
	NOR	8.531	59.584	-1.180	0.901
	MUX	12.751	82.992	-1.057	0.000
HPC2	AND	28.705	192.584	-3.320	0.138
	NAND	28.759	192.584	-3.253	0.138
	OR	29.553	192.584	-0.552	0.000
	NOR	29.414	192.052	0.202	0.139
	MUX	17.403	103.208	-3.113	-0.513
HPC3	AND	10.409	54.796	-3.529	-0.962
	NAND	10.377	54.796	-3.792	-0.962
	OR	10.918	54.530	3.774	-0.966
	NOR	10.830	53.200	2.833	-3.382
	MUX	14.717	70.224	-5.230	0.000

4.2.1 Best Fitness Selection

The best fitness selection method, in contrast to the tournament selection method, offers solutions with distinct advantages and disadvantages. We observe two main shortcomings with the best fitness selection approach: a lack of expansivity or exploration within the solution space and limited improvements. As illustrated in Figure 7, the best fitness selection method tends to reach convergence early, sometimes resulting in suboptimal solutions. While a local optimum may be close to a global optimum, this is generally not valid for such a complex solution space. To fully exploit the benefits of GAs, it is crucial to have a selection method that can explore obscure points of the solution space. As shown in Table 4, the best fitness selection method demonstrates improvements in power for most gadgets, though improvements in the area are severely limited. Hence, as presented in the next subsection, we evaluate the performance of GA equipped with tournament-based selection.

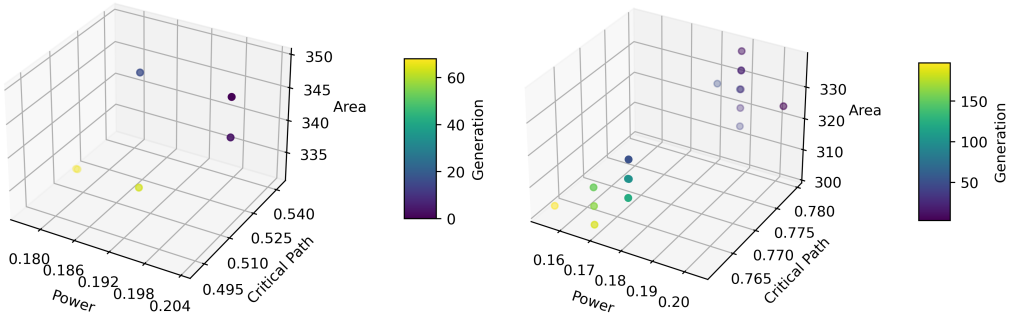

Figure 7: Search space exploration for best fitness selection (left) vs. tournament selection (right) on NOR HPC3 gadget. The latter provides an expanded search of the solution space.

4.2.2 Tournament-based Selection

For the tournament-based selection method, we derived various parameters empirically to maximize its efficacy. As shown in Table 2, we consistently select 10 individuals to participate in the tournament based on a random selection from the initial population of 100. We select the best individual among the 10 randomly chosen individuals 40 times. For each generation, we find the best fit among the 40 "tournament winners". The 40 newly created offspring and some individuals from the previous generation (elitism) form the new population for the next generation. Within the tournament-based selection framework, we compare two possible options for mutation, namely random (uniform) and Gaussian mutation. While the mutation parameters are drawn from a uniform distribution in the random mutation scenario, the Gaussian-based mutation aims to enhance exploration by introducing variability and controlled randomness from a Gaussian distribution [Hin95]. As circuit components' variation could follow a Gaussian distribution, we hypothesized that this approach would help diversify the population, ensure smoother convergence, and present stronger candidates.

First, we studied how much improvement could be achieved when applying tournament-based selection instead of selecting the best individual. We find that the tournament-based selection facilitates a more comprehensive evolution process, making it well-suited for the multi-objective nature of our optimization efforts. The tournament selection method excels over best fitness by providing a more expansive set of solutions, as seen in Figure 7. Figure 8 shows its subpar performance compared to the tournament selection method with random mutation. Table 5 details how it compares against the other methods. It demonstrates meaningful convergence towards a near-global optimum.

Gaussian-based mutation. This type of mutation can be beneficial to the design of circuits when working on real-valued parameters (e.g., power). Therefore, we examined whether this type of mutation can be applied when designing gadgets. We found that the results obtained for this were not optimal; see Table 6. At best, the averaged results are comparable to those from the randomized mutation method within the tournament-based framework, noting fewer steps to reach convergence. This can be explained by the nature of our circuit representation, where the topology or structure of the gadgets, such as gate types or wire links, are mutated. As a result, although Gaussian-based mutation provides some help in reducing the steps to convergence, it does not show a significant improvement over the random mutation implementation.

Table 5: Improvements achieved through randomized mutation under tournament selection. A negative value indicates reduced overhead.

Gadget Type	Logic Gate	Power w/o PRNG (μW)	Area w/o PRNG (μm^2)	Power Reduction [%]	Area Reduction [%]
COMAR	AND	14.781	67.032	-3.174	1.205
	NAND	15.267	66.234	0.009	0.000
	OR	15.283	66.234	-0.012	0.000
	NOR	15.769	67.032	3.185	1.205
	MUX	39.974	183.806	-1.019	-0.289
HPC1	AND	8.287	58.520	-1.271	0.000
	NAND	8.297	58.520	-1.274	0.000
	OR	8.534	57.988	-1.260	-1.802
	NOR	8.550	58.786	-0.961	-0.450
	MUX	12.572	82.726	-2.442	-0.321
HPC2	AND	18.205	165.720	-3.179	-1.383
	NAND	29.579	192.584	-0.495	0.138
	OR	29.710	192.710	-0.025	0.065
	NOR	29.181	193.914	-0.590	1.110
	MUX	17.509	103.474	-2.526	-0.256
HPC3	AND	9.152	51.870	-15.180	-6.250
	NAND	10.532	53.732	-2.353	-2.885
	OR	9.631	53.998	-8.460	-1.932
	NOR	10.217	54.530	-2.983	-0.966
	MUX	14.542	63.042	-6.358	-10.227

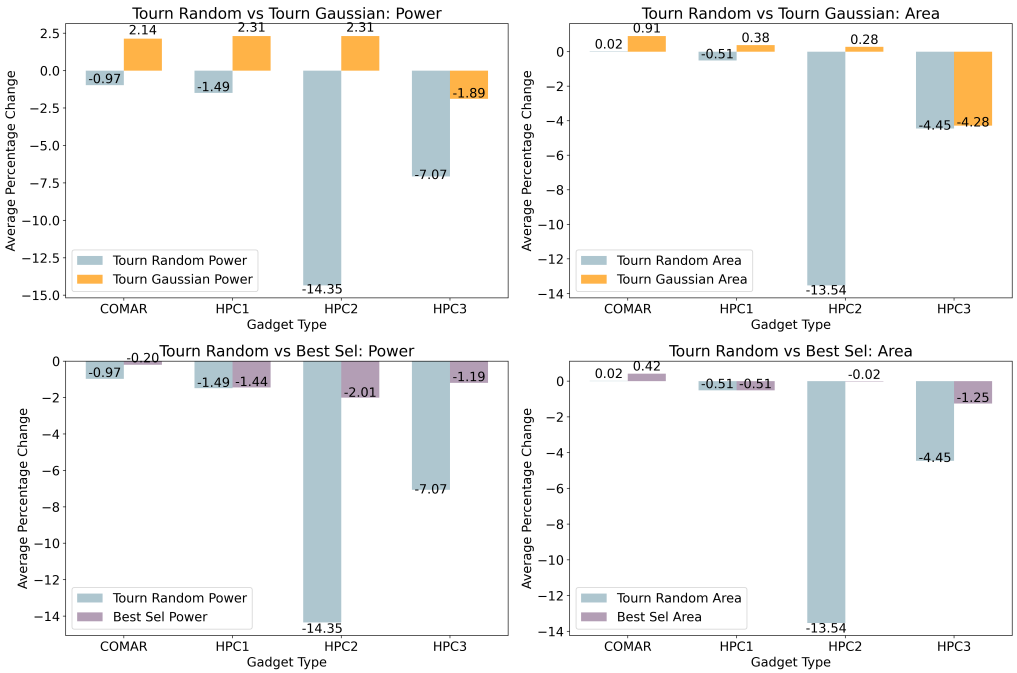

Figure 8: Comparison of the average performances of the three main evolutionary methods (tournament random, tournament Gaussian, and best selection)

4.3 Power and Area Optimization Trends

The optimization trends for randomized mutation under tournament selection, identified as the best-performing method, show that HPC2 and HPC3 gadgets are more viable for GA-based optimization. As shown in Table 5, the best-performing gadgets, specifically the AND gate, are optimized by almost 70% in both cases without losing PINI security. HPC2 gadgets initially have 75 cells, and optimization reduces this to less than 45 cells on average, achieving best-case gate combinations (similar to technology mapping) that are formally equivalent and PINI secure. In contrast, HPC3 gadgets have reduced latency

Table 6: Improvements achieved through Gaussian-based mutation under tournament selection [%]. A negative value indicates reduced overhead.

Gadget	Logic Gate	Power w/o PRNG (μW)	Area w/o PRNG (μm^2)	Power Reduction [%]	Area Reduction [%]
COMAR	AND	15.252	66.234	-0.083	0.000
	NAND	15.794	67.032	3.460	1.205
	OR	15.850	67.032	3.698	1.205
	NOR	15.988	67.830	4.620	2.410
	MUX	39.974	183.806	-1.019	-0.289
HPC1	AND	8.395	58.520	0.013	0.000
	NAND	8.917	59.318	6.101	1.364
	OR	9.021	59.584	4.372	0.901
	NOR	8.748	59.584	1.334	0.901
	MUX	12.853	81.928	-0.265	-1.282
HPC2	AND	29.690	192.318	0.000	0.000
	NAND	29.484	193.914	-0.816	0.830
	OR	29.717	192.584	0.000	0.000
	NOR	29.324	192.850	-0.102	0.555
	MUX	17.957	103.740	-0.031	0.000
HPC3	AND	10.765	55.328	-0.231	0.000
	NAND	9.949	53.200	-7.755	-3.846
	OR	10.539	54.530	0.173	-0.966
	NOR	10.231	46.550	-2.851	-15.459
	MUX	15.720	69.426	1.229	-1.136

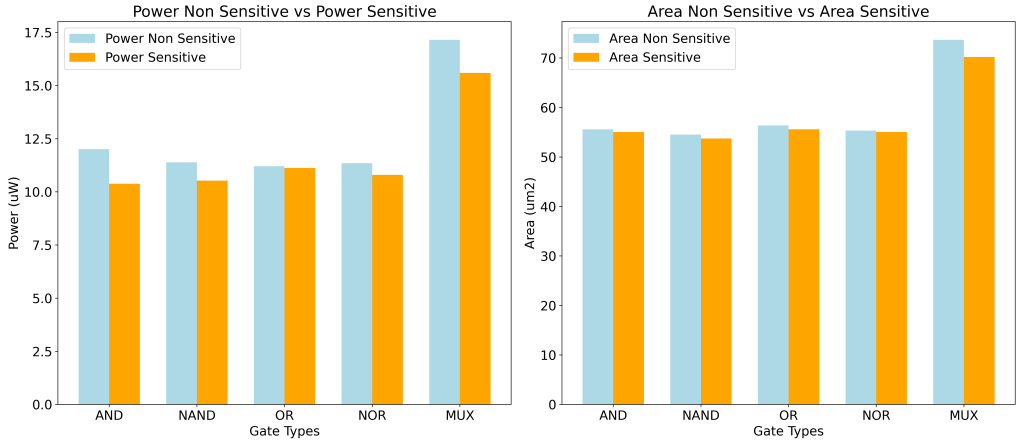

Figure 9: Comparison of tournament-based random mutation for HPC3 gadgets with 40% crossover rate and 40% mutation rate

requirements compared to HPC2, with less attention required to register combinations to maintain latency and functionality. We find consistent success in reducing both power and area.

It is important to note that this method is unsupervised, aiming to find a near-global optimum. While a more exhaustive search could yield better results, it must be performed within reasonable bounds that can be inserted into an EDA framework. COMAR and HPC1 gadgets demonstrate significant optimization restrictions, both for the mutation tournament and Gaussian mutation implementation. The combination of larger required randomness and two-stage latency for COMAR and HPC1 requires a more nuanced optimization approach, which may be explored in future work. Additionally, both HPC1 and COMAR gadgets struggled with optimization for power and area using our ABC FRAG method, as reported in Section 4.1 and Appendix A.

4.4 Sensitivity Analysis

We empirically observe that initializing single-point crossover on nodes with high switching activity yields better designs compared to randomly identifying nodes. In Figure 9, we present the results of random single-point crossover against high switching single-point crossover on the HPC3 gadgets using a top-performing GA configuration. The results demonstrate that implementing sensitivity optimization provides marginally better solutions for both power and area. These selected nodes, which correspond to areas of high activity, significantly impact the optimization of the design's success or failure. By concentrating optimization techniques on these nodes, the algorithm can effectively approach a near-global optimum without the need for exhaustive exploration of the entire search space.

4.5 Retaining PINI Security

Our framework consistently upholds security requirements for each gadget. In this sense, none of the solutions generated by our framework compromises their PINI security requirement. This focus on maintaining high-security standards aligns with the primary goals of our tool, ensuring resilience even when optimization opportunities are absent. While further stages of the EDA process may present potential opportunities for additional optimization, maintaining security remains paramount. Consequently, within our pipeline, an optimized design cannot always be found. If a new gadget cannot be generated, the initial gadget becomes the default solution, ensuring that side-channel resilience is prioritized over purely optimization-focused solutions.

4.6 Cryptographic Benchmarks Composed of Optimized Gadgets

The next requirement to address is whether the combination of individual gadget improvements results in significant area and power reductions when applied to larger cryptographic circuits, thereby enhancing their practical relevance. To this end, we take into account an AES design used in the most relevant work, COMPRESS [CGM+24]. The design is a 32-bit implementation core without the PRNG. This core uses the Boyar-Peralta [BP12] S-box optimized using the COMPRESS tool. The new S-box has lower latency and area compared to the HPC2 version created by AGEMA. The AES core uses 4 S-box-based serial implementations where the S-box components are shared by the rounds and the key scheduling module. Due to improved S-box latency, [CGM+24] has further improved the control of the original AES design. The overall optimization in terms of latency, clock cycles, and control has not been changed by us (for more details, refer to [CGM+24]). The other design used for experiments is the Skinny S-box design. Our tools have not changed the Skinny S-box design either; rather, only the gadget is optimized.

For designs like AES and Skinny S-box, we first improve gadgets and then use the optimized version in the circuit. The circuit is compiled using a Synopsys design compiler with restricted gates and drive strengths to be compatible with the VERICA tool. This allows us to test the exact design with VERICA for security analysis. For the non-optimized design (called "Base" in Table 7), we provide synthesis results using the original designs provided in the artifact of [CGM+24]. We emphasize that we computed the results for both the Base and Optimized versions of the circuits due to the difference in tools used for synthesis and security verification (VERICA) in our paper and [CGM+24].

Here, for the sake of conciseness, we report the results only for the MSKregEn gadget, which demonstrates a notable improvement compared to all the gadgets used by the Skinny S-box and AES design. For all other gadgets, the optimized versions delivered by our tool are similar to the gadgets as presented in COMPRESS. We stress that we did not feed COMPRESS gadgets to our tool, but our tool created similar gadgets as optimized

Table 7: Results of optimization of masked circuits. Base implementations are provided by COMPRESS [CGM+24]. Our tool was used to optimize the MSKregEn gadget in the COMPRESS library. Optimized circuits involved our optimized version of MSKregEn.

Design		d	Power w/o PRNG (μW)	Area w/o PRNG (μm^2)	Timing (Slack)
Skinny S-box Serialized	Base	2	261.018	684.683	2.39
	Optimized	2	262.468	653.029	2.39
AES 32 bit	Base	2	6940.000	17861.367	1.0
	Optimized	2	6110.000	16760.127	1.01

by COMPRESS. We replaced the MSKregEn gadget implementation with our optimized version. In addition to the reduced area and power reduction by 44% and 39%, respectively, the optimized MSKregEn gadget used only 3 cells instead of 4. The results of this can be seen in Table 7, for the AES 32-bit core, there is a 7.2% and 12% improvement in area and power, respectively. For the Skinny S-box design, the power is the same, but the area is reduced by about 5%. The difference between the two designs is due to the sizes of the circuits and the number of MSKregEn modules in the larger design.

5 Discussion and Future Work

Delay Optimization. Our framework provides a unique optimization path for secure designs. Delay and latency optimization proved challenging in our framework, primarily due to the need for specialized approaches that go beyond the capabilities of GAs. In rare cases, we observed reduced critical path delay values at the expense of power and/or area consumption. Effective optimization of delay often demands precise, timing-aware techniques, such as Pan's Algorithm [PKL98]. This algorithm is known for its ability to balance timing paths by identifying critical paths and redistributing delays, achieving a more even distribution of latency across the circuit. We are also restricted by the limitations of VERICA. As mentioned earlier, the use of only single drive strength gates in the NanGate 45nm library severely restricts the use of faster gates that would improve delay.

Limitations of string-based representation. One can utilize a rapid, string-based graph implementation, which involves converting a digraph version of the benchmark into a string representation for evolution within our pipeline. This method preserved the original structure of the graphs or circuits. However, when employing that, we encountered challenges in correctly implementing certain mutation and crossover operations, often resulting in repeating identical nodes, ultimately reducing the diversity of solutions within the search space. Therefore, we did not use this representation.

Limitations of VERICA. Two current constraints in VERICA that have an impact on the results are as follows. First, it supports the X1 drive strength by default, providing a consistent baseline for evaluation, though it may influence the PPA calculation. Second, the use of the NanGate45 library [Nan] offers a standardized foundation, but also presents an opportunity for future work to explore pre-silicon behavior across a wider range of libraries, particularly for studying effects like coupling.

To the best of our knowledge, we are the first to use evolutionary algorithms within a secure design framework. During our work, we have identified a few areas that can be researched in the future, as follows:

- **Further reduction of search space.** The evolutionary algorithm search space can be reduced by incorporating logic from tools like AGEMA to create secure circuits. Verification tools also use probing security rules to create masked circuits [RBFSG22, MM22, KSM20]. These rules could be used as a basis for improving the selection/

mutation parameters of the GA.
- **Using other verification tools.** Although VERICA is effective for our application, it has limitations, such as drive strength and library support. Future work can incorporate different tools with varied capabilities to enhance our framework.
- **Extension to FPGAs.** While our primary focus is pre-silicon design for the ASIC design flow, the increasing use of FPGA motivates the extension of our framework to include FPGA optimization. This is a challenging task as it involves incorporating FPGA software from major vendors like Xilinx.

6 Conclusion

This work demonstrates the potential of applying AI along with EDA to optimize existing gadgets and overall designs. Our focus lies on evolutionary algorithms to benefit PPA metrics. The primary objective is to demonstrate the effectiveness of these techniques in viable gates without compromising security. Our results demonstrate improvements in total power for the NanGate 45nm library and promising area improvements compared to the non-optimized counterparts. Our results also showcase a substantial reduction in total power and area metrics, achieving up to 68% reduction in some cases, compared to unconstrained synthesis of the masked gadgets.

A Optimization using Technology Mapping and FRAIGs

In Section 4.1, we briefly outlined the results of applying ABC tool. Here, we present more detailed results.

Table 8: Reported results from Synopsys Design Compiler and VERICA using proposed framework. UB- Cannot by translated by Design Compiler, UV- Cannot be parsed VERICA, UFS- Unuseable from source

Gadget	Logic Gate	POWER (μW)			AREA (μm^2)			PINI-SECURE		
		NO	ORE	ONRE	NO	ORE	ONRE	NO	ORE	ONRE
COMAR	NAND	24.26	25.85	24.25	66.23	69.16	66.23	Yes	Yes	Yes
	AND	24.26	25.88	24.25	66.23	69.16	66.23	Yes	Yes	Yes
	OR	24.33	25.84	24.34	66.23	69.16	66.23	Yes	Yes	Yes
	NOR	24.33	25.93	24.34	66.23	69.16	66.23	Yes	Yes	Yes
	XNOR	23.65	23.27	23.61	63.04	61.45	63.04	Yes	Yes	Yes
	MUX	79.62	UB	70.65	196.04	UB	169.71	Yes	UB	Yes
	XOR	23.66	22.85	23.61	63.04	61.45	63.04	Yes	Yes	Yes
HPC1	NAND	21.07	19.28	19.28	59.05	53.73	54.00	Yes	Yes	Yes
	AND	20.93	19.34	19.35	58.52	53.73	54.00	No	Yes	Yes
	OR	21.24	21.30	19.37	60.12	58.52	54.53	Yes	Yes	Yes
	NOR	21.09	19.40	19.42	59.58	54.26	54.53	Yes	Yes	Yes
	XNOR					UFS				
	MUX	30.23	26.61	28.55	82.99	74.75	78.47	No	Yes	Yes
	XOR					UFS				
HPC2	NAND	70.92	39.96	48.21	198.70	113.58	131.67	UV	Yes	Yes
	AND	70.75	39.97	48.15	198.70	113.58	131.67	UV	Yes	Yes
	OR	70.94	39.79	48.01	198.97	113.58	131.67	UV	Yes	Yes
	NOR	70.77	39.79	47.15	198.97	113.58	131.67	UV	Yes	Yes
	MUX	38.91	21.65	22.42	106.93	60.91	60.91	UV	Yes	Yes
	XOR					UFS				
HPC3	NAND	18.60	10.13	8.63	58.52	30.59	26.07	UV	Yes	Yes
	AND	18.72	10.15	8.68	58.52	30.59	26.07	UV	Yes	Yes
	OR	18.56	8.58	8.58	58.25	26.07	26.07	UV	Yes	Yes
	NOR	18.67	8.63	8.63	58.25	26.07	26.07	UV	Yes	Yes
	XNOR					UFS				
	MUX	25.02	10.05	13.55	73.95	31.12	39.90	UV	Yes	Yes
	XOR					UFS				
GHPC	PRESENT-TI	2910.00	UB	108.28	494.76	UB	498.22	Yes	UB	Yes
	AES-TI	2910.00	UB	3560.00	5121.56	UB	6875.30	Yes	UB	Yes
	AES-DOM	534.05	UB	1240.00	2334.42	UB	3954.36	Yes	UB	Yes

References

[ASF08] Fabricio Vivas Andrade, Leandro M Silva, and Antonio O Fernandes. Improving sat-based combinational equivalence checking through circuit preprocessing. In *2008 IEEE International Conference on Computer Design*, pages 40–45. IEEE, 2008.

[BBD+15] Gilles Barthe, Sonia Belaïd, François Dupressoir, Pierre-Alain Fouque, Benjamin Grégoire, and Pierre-Yves Strub. Verified proofs of higher-order masking. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 457–485. Springer, 2015.

[BBD+16] Gilles Barthe, Sonia Belaïd, François Dupressoir, Pierre-Alain Fouque, Benjamin Grégoire, Pierre-Yves Strub, and Rébecca Zucchini. Strong non-interference and type-directed higher-order masking. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, pages 116–129, 2016.

[BBP+16] Sonia Belaïd, Fabrice Benhamouda, Alain Passelègue, Emmanuel Prouff, Adrian Thillard, and Damien Vergnaud. Randomness complexity of private circuits for multiplication. In *Advances in Cryptology–EUROCRYPT 2016: 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8-12, 2016, Proceedings, Part II 35*, pages 616–648. Springer, 2016.

[BBYS22] Ileana Buhan, Lejla Batina, Yuval Yarom, and Patrick Schaumont. Sok: Design tools for side-channel-aware implementations. In *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*, pages 756–770, 2022.

[BM10] Robert Brayton and Alan Mishchenko. Abc: An academic industrial-strength verification tool. In Tayssir Touili, Byron Cook, and Paul Jackson, editors, *Computer Aided Verification*, pages 24–40, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg.

[BMN+23] Jitendra Bhandari, Likhitha Mankali, Mohammed Nabeel, Ozgur Sinanoglu, Ramesh Karri, and Johann Knechtel. Beware your standard cells! on their role in static power side-channel attacks. Cryptology ePrint Archive, Paper 2023/920, 2023. URL: https://eprint.iacr.org/2023/920.

[BP12] Joan Boyar and René Peralta. A small depth-16 circuit for the aes s-box. In *IFIP International Information Security Conference*, pages 287–298. Springer, 2012.

[BS93] Thomas Bäck and Hans-Paul Schwefel. An overview of evolutionary algorithms for parameter optimization. *Evolutionary computation*, 1(1):1–23, 1993.

[BT96] Tobias Blickle and Lothar Thiele. A comparison of selection schemes used in evolutionary algorithms. *Evolutionary Computation*, 4(4):361–394, 1996.

[BW09] Zhiguo Bao and Takahiro Watanabe. A novel genetic algorithm with cell crossover for circuit design optimization. In *2009 IEEE International Symposium on Circuits and Systems*, pages 2982–2985. IEEE, 2009.

[CGLS20] Gaëtan Cassiers, Benjamin Grégoire, Itamar Levi, and François-Xavier Standaert. Hardware private circuits: From trivial composition to full verification. *IEEE TC*, 70(10):1677–1690, 2020.

[CGM+24] Gaëtan Cassiers, Barbara Gigerl, Stefan Mangard, Charles Momin, and Rishub Nagpal. Compress: Generate small and fast masked pipelined circuits. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024(3):500–529, 2024.

[CJRR99] Suresh Chari, Charanjit S. Jutla, Josyula R. Rao, and Pankaj Rohatgi. Towards sound approaches to counteract power-analysis attacks. In *CRYPTO*, 1999.

[COM] COMPRESS: Generate Small and Fast Masked Pipelined Circuits - Projects - Graz University of Technology. URL: https://graz.elsevierpure.com/en/publications/compress-generate-small-and-fast-masked-pipelined-circuits/projects/.

[CS20a] Gaëtan Cassiers and François-Xavier Standaert. Trivially and efficiently composing masked gadgets with probe isolating non-interference. *IEEE TIFS*, 15:2542–2555, 2020.

[CS20b] Gaetan Cassiers and Francois Xavier Standaert. Trivially and Efficiently Composing Masked Gadgets with Probe Isolating Non-Interference. *IEEE Transactions on Information Forensics and Security*, 15:2542–2555, 2020. doi:10.1109/TIFS.2020.2971153.

[DAPM00] Kalyanmoy Deb, Samir Agrawal, Amrit Pratap, and Tanaka Meyarivan. A fast elitist non-dominated sorting genetic algorithm for multi-objective optimization: Nsga-ii. In *Parallel Problem Solving from Nature PPSN VI: 6th International Conference Paris, France, September 18–20, 2000 Proceedings 6*, pages 849–858. Springer, 2000.

[DCEM18] Thomas De Cnudde, Maik Ender, and Amir Moradi. Hardware masking, revisited. *IACR TCHES*, pages 123–148, 2018.

[DM13] Dipankar Dasgupta and Zbigniew Michalewicz. *Evolutionary algorithms in engineering applications*. Springer Science & Business Media, 2013.

[DN20] Siemen Dhooghe and Svetla Nikova. My gadget just cares for me-how nina can prove security against combined attacks. In *Cryptographers' Track at the RSA Conference*, pages 35–55. Springer, 2020.

[DPAM02] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and TAMT Meyarivan. A fast and elitist multiobjective genetic algorithm: Nsga-ii. *IEEE transactions on evolutionary computation*, 6(2):182–197, 2002.

[DWZG18] Haiming Du, Zaichao Wang, WEI Zhan, and Jinyi Guo. Elitism and distance strategy for selection of evolutionary algorithms. *IEEe Access*, 6:44531–44541, 2018.

[EM11] Tariq El Motassadeq. Ccs vs nldm comparison based on a complete automated correlation flow between primetime and hspice. In *2011 Saudi International Electronics, Communications and Photonics Conference (SIECPC)*, pages 1–5. IEEE, 2011.

[FDRG+12] Félix-Antoine Fortin, François-Michel De Rainville, Marc-André Gardner Gardner, Marc Parizeau, and Christian Gagné. Deap: Evolutionary algorithms made easy. *The Journal of Machine Learning Research*, 13(1):2171–2175, 2012.

[FGDP+18] Sebastian Faust, Vincent Grosso, Santos Merino Del Pozo, Clara Paglialonga, and François-Xavier Standaert. Composable masking schemes in the presence of physical defaults & the robust probing model. *IACR TCHES*, pages 89–120, 2018.

[FSG23] Jakob Feldtkeller, Pascal Sasdrich, and Tim Güneysu. Challenges and opportunities of security-aware eda. *ACM Transactions on Embedded Computing Systems*, 22(3):1–34, 2023.

[GF21] Ana V. ović, Fatemeh Ganji, and Domenic Forte. Circuit masking: From theory to standardization, a comprehensive survey for hardware security researchers and practitioners. *ArXiv*, 2021.

[GM17] Hannes Groß and Stefan Mangard. Reconciling $d + 1$ masking in hardware and software. In *CHES*, pages 115–136. Springer, 2017.

[GMK16] Hannes Gross, Stefan Mangard, and Thomas Korak. Domain-oriented masking: Compact masked hardware implementations with arbitrary protection order. In *ACM TIS*, page 3, New York, NY, USA, 2016. ACM.

[GMK17] Hannes Groß, Stefan Mangard, and Thomas Korak. An efficient side-channel protected aes implementation with arbitrary protection order. In *Cryptographers' Track at the RSA Conference*, pages 95–112. Springer, 2017.

[GMO01] Karine Gandolfi, Christophe Mourtel, and Francis Olivier. Electromagnetic analysis: Concrete results. In *CHES*, pages 251–261. Springer, 2001.

[GS20] Giorgio Guariso and Matteo Sangiorgio. Improving the performance of multiobjective genetic algorithms: An elitism-based approach. *Information*, 11(12):587, 2020.

[GSF13] Vincent Grosso, François-Xavier Standaert, and Sebastian Faust. Masking vs. multiparty computation: how large is the gap for aes? In *Cryptographic Hardware and Embedded Systems-CHES 2013: 15th International Workshop, Santa Barbara, CA, USA, August 20-23, 2013. Proceedings 15*, pages 400–416. Springer, 2013.

[Hin95] Robert Hinterding. Gaussian mutation and self-adaption for numeric genetic algorithms. In *Proceedings of 1995 IEEE International Conference on Evolutionary Computation*, volume 1, page 384. IEEE, 1995.

[ISW03] Yuval Ishai, Amit Sahai, and David Wagner. Private circuits: Securing hardware against probing attacks. In *CRYPTO*, pages 463–481. Springer, 2003.

[JG99] Domagoj Jakobović and Marin Golub. Adaptive genetic algorithm. *Journal of computing and information technology*, 7(3):229–235, 1999.

[KJJ99] Paul Kocher, Joshua Jaffe, and Benjamin Jun. Differential power analysis. In *CRYPTO*, pages 388–397. Springer, 1999.

[KM22a] David Knichel and Amir Moradi. Composable gadgets with reused fresh masks: First-order probing-secure hardware circuits with only 6 fresh masks. *IACR TCHES*, 2022(3):114–140, Jun. 2022. URL: https://tches.iacr.org/index.php/TCHES/article/view/9696.

[KM22b] David Knichel and Amir Moradi. Low-latency hardware private circuits. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 1799–1812, 2022.

[KMMS21] David Knichel, Amir Moradi, Nicolai Müller, and Pascal Sasdrich. Automated generation of masked hardware. *Cryptology ePrint Archive*, 2021.

[KS22] Pantea Kiaei and Patrick Schaumont. Soc root canal! root cause analysis of power side-channel leakage in system-on-chip designs. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 751–773, 2022.

[KSM20] David Knichel, Pascal Sasdrich, and Amir Moradi. Silver–statistical independence and leakage verification. In *Asiacrypt*, pages 787–816. Springer, 2020.

[KSM22] David Knichel, Pascal Sasdrich, and Amir Moradi. Generic hardware private circuits: Towards automated generation of composable secure gadgets. *IACR TCHES*, pages 323–344, 2022.

[MCBP06] Alan Mishchenko, Satrajit Chatterjee, Robert Brayton, and Peichen Pan. Integrating logic synthesis, technology mapping, and retiming. 01 2006.

[MCJB05] Alan Mishchenko, Satrajit Chatterjee, Roland Jiang, and Robert K. Brayton. Fraigs: A unifying representation for logic synthesis and verification. 2005. URL: https://api.semanticscholar.org/CorpusID:9209313.

[MCS22] Charles Momin, Gaëtan Cassiers, and François-Xavier Standaert. Handcrafting: Improving automated masking in hardware with manual optimizations. In *COSADE*, pages 257–275. Springer, 2022.

[MM22] Nicolai Müller and Amir Moradi. Prolead: A probing-based hardware leakage detection tool. *IACR TCHES*, 2022(4):311–348, Aug. 2022. URL: https://tches.iacr.org/index.php/TCHES/article/view/9822, doi:10.46586/tches.v2022.i4.311-348.

[MMG+23] N. Muller, S. Meschkov, D. E. Gnad, M. B. Tahoori, and A. Moradi. Automated masking of fpga-mapped designs. In *FPL*, pages 79–85, Los Alamitos, CA, USA, sep 2023. IEEE Computer Society. URL: https://doi.ieeecomputersociety.org/10.1109/FPL60245.2023.00019, doi:10.1109/FPL60245.2023.00019.

[MS08] Joao Marques-Silva. Practical applications of boolean satisfiability. In *2008 9th International Workshop on Discrete Event Systems*, pages 74–80. IEEE, 2008.

[MZM07] Natasa Miskov-Zivanov and Diana Marculescu. Soft error rate analysis for sequential circuits. In *2007 Design, Automation & Test in Europe Conference & Exhibition*, pages 1–6, 2007. doi:10.1109/DATE.2007.364500.

[MZM08] Natasa Miskov-Zivanov and Diana Marculescu. Modeling and optimization for soft-error reliability of sequential circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 27(5):803–816, 2008.

[MZS+23] Haiping Ma, Yajing Zhang, Shengyi Sun, Ting Liu, and Yu Shan. A comprehensive survey on nsga-ii for multi-objective optimization and applications. *Artificial Intelligence Review*, 56(12):15217–15270, 2023.

[Nan] Nangate. 45nm. [Online]https://tilos-ai-institute.github.io/MacroPlacement/Enablements/NanGate45/[Accessed:Apr.7,2025], year=2025.

[PKL98] Peichen Pan, Arvind K Karandikar, and CL Liu. Optimal clock period clustering for sequential circuits with retiming. *IEEE transactions on computer-aided design of integrated circuits and systems*, 17(6):489–498, 1998.

[RBFSG22] Jan Richter-Brockmann, Jakob Feldtkeller, Pascal Sasdrich, and Tim Güneysu. Verica - verification of combined attacks: Automated formal verification of security against simultaneous information leakage and tampering. *IACR TCHES*, 2022(4):255–284, Aug. 2022.

[RBN+15] Oscar Reparaz, Begül Bilgin, Svetla Nikova, Benedikt Gierlichs, and Ingrid Verbauwhede. Consolidating masking schemes. In *CRYPTO*, pages 764–783. Springer, 2015.

[RNJ05] Carlo R Raquel and Prospero C Naval Jr. An effective use of crowding distance in multiobjective particle swarm optimization. In *Proceedings of the 7th Annual conference on Genetic and Evolutionary Computation*, pages 257–264, 2005.

[Sam76] Jeffrey R Sampson. Adaptation in natural and artificial systems (john h. holland), 1976.

[Sed12] Philip Sedgwick. Pearson's correlation coefficient. *Bmj*, 345, 2012.

[Ses06] Aravind Seshadri. A fast elitist multiobjective genetic algorithm: Nsga-ii. *MATLAB Central*, 182:182–197, 2006.

[SM21] Aein Rezaei Shahmirzadi and Amir Moradi. Re-consolidating first-order masking schemes: Nullifying fresh randomness. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 305–342, 2021.

[SSM15] Trailokya Nath Sasamal, Ashutosh Kumar Singh, and Anand Mohan. Reversible logic circuit synthesis and optimization using adaptive genetic algorithm. *Procedia Computer Science*, 70:407–413, 2015.

[Syn23] Synopsys. Design compiler concurrent timing, area, power, and test optimization, 2023. URL: https://www.synopsys.com/implementation-and-signoff/rtl-synthesis-test/dc-ultra.html.

[TAM93] Vivek Tiwari, Pranav Ashar, and Sharad Malik. Technology mapping for lower power. In *DAC*, pages 74–79, 1993.

[Tri03] Elena Trichina. Combinational logic design for aes subbyte transformation on masked data. *Cryptology EPrint Archive*, 2003.

[VPS21] Shanu Verma, Millie Pant, and Vaclav Snasel. A comprehensive review on nsga-ii for multi-objective combinatorial optimization problems. *IEEE access*, 9:57757–57791, 2021.

[WCC09] Laung-Terng Wang, Yao-Wen Chang, and Kwang-Ting Tim Cheng. *Electronic design automation: synthesis, verification, and test*. Morgan Kaufmann, 2009.

[WFP+23] Lixuan Wu, Yanhong Fan, Bart Preneel, Weijia Wang, and Meiqin Wang. Automated generation of masked nonlinear components: From lookup tables to private circuits. [Online] https://eprint.iacr.org/2023/831 [Accessed: Nov.20, 2023], 2023.

[WGK13] Clifford Wolf, Johann Glaser, and Johannes Kepler. Yosys-a free verilog synthesis suite. 2013. URL: https://api.semanticscholar.org/CorpusID:202611483.

[ZK00] Byoung-Tak Zhang and Jung-Jib Kim. Comparison of selection methods for evolutionary optimization. *Evolutionary optimization*, 2(1):55–70, 2000.